

# Constructible Proofs for the Arithmetic Kakeya Conjecture

Structure, obstructions, and the axis that can pay

Jeromie Beasley

2 September 2026

## Abstract

The Epoch FrontierMath open problem *The Arithmetic Kakeya Conjecture* asks for a finite dilate set  $X \subset \mathbb{Z}^2$ , an  $X$ -constructible tower, and a forcing pair whose score  $(m + |R|)/(n - |T|)$  is at most 1.675, below the record  $\gamma = 1.6751308\dots$  of Katz and Tao that has stood since 2002. This note is the complete account of an attempt on it. Nothing here improves the bound. What is established instead is the shape of the space. The problem's two formulations are provably different classes of objects, and the machine-checkable one — which is the submission format — is the smaller. The cut argument yields a rank obstruction holding for every subset of the unknown vertices, with a per-vertex corollary and a seeding lemma governing the first vertex to fall. A score is a ratio of small integers, so any winner with fewer than forty free vertices must reach  $5/3$ , and for the sums–differences set the proven lower bound eliminates most sizes outright. Iterating a construction provably never helps: edge density adds under tower products, so amortisation can come only from a bounded generator set serving a growing graph — more dilates, not bigger graphs. An integer symmetry group carries any object through an infinite orbit at fixed score. The minimum generator count is not a function of the label multiset but of the arrangement. The Shannon-inequality cone sees nothing below the trivial bound. Three independent forcing engines agree on every instance tested, a seeded search recovers the published objects unaided and finds a new  $11/6$  object beside them, and several regions are closed exactly. The live search is the cell that the structure points at: the Katz–Tao ladder with one generator removed, over richer dilates. Two thirds of one orientation is closed at height two with nothing found; the rest is resumable from the scripts in Appendix A.

## 0. Claims and grades

[T] proved here    [V] verified by exact computation    [C] cited    [O] open

| Claim                                                                                                                            | Grade        |
|----------------------------------------------------------------------------------------------------------------------------------|--------------|
| The verifiable and intuitive set-ups are different classes; the verifiable one is the submission format                          | [T], [C]     |
| Subset rank obstruction: $\text{rank } L(S) = 2$ for every nonempty $S \subseteq V \setminus T_0$                                | [T]          |
| Per-vertex corollary and the bound $ R  \geq \sum_e \max(0, 2 - \text{rank}_e)$                                                  | [T]          |
| Seeding lemma: from $T_0 = \emptyset$ the first vertex needs rank 2 from its own generators                                      | [T]          |
| Under 40 free vertices any score $\leq 67/40$ is $\leq 5/3$                                                                      | [T]          |
| Sums–differences: eleven sizes admit no object, stable for any floor in $(16/9, 25/14]$                                          | [T] with [C] |
| Products cannot amplify: $m/n$ adds under tower concatenation                                                                    | [T]+[V]      |
| Forcing decomposes over connected components                                                                                     | [V]          |
| Integer symmetry group at fixed score, two one-parameter families                                                                | [T]+[V]      |
| The minimum generator count depends on the label arrangement, not the multiset                                                   | [V]          |
| Shannon LP with two copies proves nothing below 2                                                                                | [V]          |
| Three engines agree: 6000 random instances, no false negatives                                                                   | [V]          |
| Seeded search recovers figure 2 unaided and finds a second 11/6 object                                                           | [V]          |
| Regions closed (Sec. 8): Goal 2 at $n -  T  = 5$ , $n \leq 6$ ; Goal 1 at $n \leq 5$ and at $n = 6$ on both ladders and the path | [V]          |
| Record hunt: 33 of 49 sub-chunks of $d = (2, 3)$ closed at $ R  = 3$ , height 2                                                  | [V]          |
| Novelty of the obstruction against Katz 2006                                                                                     | [O]          |
| Whether the two formulations achieve the same scores                                                                             | [O]          |

## 1 The problem

Say  $\text{AK}(\alpha)$  holds if for every  $\epsilon > 0$  there is a finite  $X \subset \mathbb{Z}^2$ , with  $x_1 + x_2 \neq 0$  for all  $\mathbf{x} \in X$ , such that  $|(1, -1) \cdot G| \ll_\epsilon \max_{\mathbf{x} \in X} |\mathbf{x} \cdot G|^{\alpha+\epsilon}$  for all finite  $G \subset \mathbb{Z}^2$ . Then every Kakeya set in  $\mathbb{R}^d$  has Hausdorff dimension at least  $\alpha^{-1}d + 1 - \alpha^{-1}$ , and the arithmetic Kakeya conjecture is  $\text{AK}(1)$ . Bourgain proved  $\text{AK}(25/13)$  [3]; Katz and Tao proved  $\text{AK}(11/6)$  and, with one more projection,  $\text{AK}(7/4)$  [6]; then  $\text{AK}(\gamma)$  with  $\gamma = 1.6751308\dots$  the largest root of  $x^3 - 4x + 2$  [7]. Katz showed the elementary method cannot pass  $3/2$  [5].

A constructible proof is an  $X$ -labelled graph  $G$  on  $n$  vertices and  $m$  edges with a forcing pair  $(R, T)$ :  $R$  a set of single-vertex generators  $\mathbf{x}g$ ,  $T$  a set of vertices known at the outset, such that repeatedly adding edge generators  $\mathbf{v}g_1 - \mathbf{v}g_2$ , taking  $\mathbb{Z}$ -combinations, and adding to  $T$  any vertex  $e$  at which some module element is  $(a, -a)$ ,  $a \neq 0$ , while vanishing outside  $T \cup \{e\}$ , eventually gives  $T = V$ . Its score is  $(m + |R|)/(n - |T|)$ , and a constructible proof of score  $\alpha$  establishes  $\text{AK}(\alpha)$ .

### Two goals

**Goal 1.** Any finite  $X$ ; score  $\leq 1.675 = 67/40$ .

**Goal 2.**  $X = \{(1, 0), (0, 1), (1, 1)\}$  exactly; score strictly below  $11/6$ . This is the sums–differences constant, and its true value is pinned between Lemm’s lower bound  $1.77898$  [8] and  $11/6$ .

The separation governs everything. Lemm’s bound is on the true constant, so no construction on the three-slope set can ever reach  $1.675$ . Goal 1 requires dilates beyond those three, and the  $\gamma$

construction is known to need  $|X|$  growing without bound. The bound also supplies a permanent free control: any completing object on the three-slope set scoring below 1.77898 would contradict a theorem, so its appearance means the verifier is broken.

## The origin

Besicovitch was not working on needles. In 1917 he was asking whether, for a Riemann integrable  $f$  on the plane, there is a coordinate system in which the iterated integral exists, and he reduced it to a measure-zero planar set containing a segment in every direction [2]. His counterexample takes the points of that set with at least one *rational* coordinate. The obstruction is arithmetic, not geometric, and the additive-combinatorial formulation is closer to that origin than the needle picture is. [C]

## 2 Two formulations, two classes

The problem is stated twice: an iterative definition for a reader, and a tower encoding for a machine. The statement calls them equivalent. They are not the same class of labelled objects.

The iterative form builds  $H'$  from  $k+1$  copies of a constructible  $H$  by, for chosen vertex sets  $X_i$ , either *identifying* corresponding vertices of consecutive copies or joining them with an edge carrying its own label. The tower encoding fixes  $d_1, \dots, d_k$ , takes vertices  $(e_1, \dots, e_k)$  with  $1 \leq e_i \leq d_i$ , and supplies  $f_i : d_1 \times \dots \times d_{i-1} \times (d_i - 1) \rightarrow X$ , where  $f_i(e_1, \dots, e_i)$  labels the edge from  $(e_1, \dots, e_i, \mathbf{c})$  to  $(e_1, \dots, e_i + 1, \mathbf{c})$  for every suffix  $\mathbf{c}$ . One label creates a whole slab and costs the whole slab:  $n = d_1 \dots d_k$  and  $m = \sum_i \sum_{\mathbf{e}} \mathbf{1}_{f_i(\mathbf{e}) \neq 0} d_{i+1} \dots d_k$ .

**Theorem 1.** *There is a Definition-1 constructible graph that no tower represents. [T]*

*Proof.* The path  $a-b-c$  is constructible. Take two copies and identify the middle vertex. The result has 5 vertices, 4 edges, and  $b$  adjacent to  $a_0, c_0, a_1, c_1$  — degree 4. A tower on  $n$  vertices has degree at most  $2k$  with  $n = d_1 \dots d_k$ . Since 5 is prime the only tower on 5 vertices is  $d = (5)$ , a path, of maximum degree 2.  $\square$

The mechanism is identification, which the tower encoding has no way to express since  $n$  is always the full product. The tower encoding is also orientation dependent: Katz–Tao’s six-vertex configuration has two parallel rails with different labels in the same construction step, expressible at  $d = (2, 3)$  where they sit in separate prefixes and inexpressible at  $d = (3, 2)$  where they share a slab (Figure 1).

Two things follow. The tower encoding is Epoch’s own submission format — the answer is  $d_1 \dots d_k$  and the  $f_i$  dictionaries — so the restriction belongs to the problem, not to any search over it, and an empty tower search is exactly as strong as the problem allows. And the claim of equivalence in the problem statement is false at the level of objects; whether the two classes achieve the same *scores* is open. [O]

## 3 The subset rank obstruction

Write  $s(\mathbf{x}) = x_1 + x_2$ . Every label has  $s \neq 0$  by hypothesis, and the forbidden direction  $(a, -a)$  is exactly where  $s$  vanishes. The rules forbid carrying that direction and cannot forbid manufacturing it as a difference; that is the whole tension.

**Theorem 2.** *Let  $(R, T_0)$  be a forcing pair for a labelled graph on  $V$ . For  $S \subseteq V$  put  $L(S) = \text{span}_{\mathbb{Z}}\{\mathbf{x} : \mathbf{x}g \in R, g \in S\} + \text{span}_{\mathbb{Z}}\{\mathbf{x}_E : E \text{ crosses } \partial S\}$ . If forcing completes then  $\text{rank } L(S) = 2$  for every nonempty  $S \subseteq V \setminus T_0$ . [T]*

*Proof.* Let  $e$  be the first vertex of  $S$  to enter  $T$ ; at that moment  $S$  lies inside the unknown set. Operation 2 supplies  $w$  supported on  $T \cup \{e\}$  with  $w(e) = (a, -a)$ ,  $a \neq 0$ . Write  $w$  as an integer combination of vertex generators and edge generators and sum its columns over  $S$ . An edge generator with both endpoints in  $S$  contributes  $\mathbf{x}_E - \mathbf{x}_E = 0$ ; one with neither contributes nothing; one crossing  $\partial S$  contributes  $\pm \mathbf{x}_E$ . A vertex generator contributes its label if its vertex is in  $S$ . Since  $S \cap (T \cup \{e\}) = \{e\}$ , the sum is  $w(e)$ , so  $w(e) \in L(S)$ . A rank-1 lattice spanned by a label with  $s \neq 0$  contains no nonzero vector with  $s = 0$ .  $\square$

**Corollary 3** (per vertex). *Every  $e \in V \setminus T_0$  needs  $\text{rank}(\{\text{generators at } e\} \cup \{\text{labels incident to } e\}) = 2$ , hence  $|R| \geq \sum_{e \notin T_0} \max(0, 2 - \text{rank}(\text{incident labels at } e))$ .*

The corollary was checked on both published configurations and on 4000 random five-vertex objects: fifteen completed, none violated it (Appendix A.5). [V]

**Lemma 4** (seeding). *From  $T_0 = \emptyset$ , the first vertex to fall needs rank 2 from its own vertex generators alone. [T]*

*Proof.* With  $T = \emptyset$  the witness  $w$  must vanish at every vertex but  $e$ . Every edge incident to  $e$  has a far endpoint at which  $w$  must vanish, and the only module elements supported on  $\{e\}$  alone are combinations of generators at  $e$  together with edge combinations that cancel at every other vertex; by Theorem 2 applied to  $S = \{e\}$  the latter contribute only incident labels, and by the vanishing at the far endpoints their coefficients are forced to zero unless cancelled by further edges, which in turn require their own cancellation, terminating only at  $e$ . So  $w(e)$  lies in the span of the generators at  $e$ .  $\square$

The lemma is why Corollary 3's bound is loose exactly where cost is highest — it credits incident labels that cannot yet be used — and it is the prune that makes the record hunt of Sec. 8 finite.

**Novelty.** Katz's proof that the method cannot pass  $3/2$  [5] is six pages, not on arXiv, and unobtainable through every route tried, including the journal's own repository, which serves a stale embargo notice. Green–Ruzsa [4] and Pohoata–Zakharov [9] do not cite it. Its subject classification is *random graphs*, which points at a probabilistic construction rather than a rank obstruction. Until it is read, Theorem 2 is *stated here; comparison with Katz 2006 pending*, and is not called new. The barrier is at  $3/2$  and the target is 1.675, so it does not block the search either way. [O]

## 4 Score granularity

**Proposition 5.** *For  $q = n - |T| < 40$  the largest value of  $(m + |R|)/q$  not exceeding  $67/40$  is  $5/3$ . [T]*

*Proof.* Let  $p = \lfloor 67q/40 \rfloor$  and suppose  $3p \geq 5q + 1$ . Then  $120p \geq 200q + 40$ , while  $p \leq 67q/40$  gives  $120p \leq 201q$ ; so  $200q + 40 \leq 201q$  and  $q \geq 40$ . Equality  $p/q = 5/3$  holds at every multiple of 3 below 40.  $\square$

So any object with fewer than forty free vertices that beats the record beats it all the way to  $5/3$  (Figure 2). For Goal 2, requiring a score strictly below  $11/6$  and at least Lemm's 1.77898 leaves  $q \in \{5, 10, 11, 14, 15, 16, 17, \dots\}$  and kills  $q \in \{1, 2, 3, 4, 6, 7, 8, 9, 12, 13, 18\}$  outright; the

list is unchanged for any floor in  $(16/9, 25/14] = (1.77778, 1.785714]$ , so it does not depend on which decimal of the lower bound is current. The smallest conceivable improvement on the sums–differences bound is  $q = 5$ ,  $m + |R| = 9$ , score  $9/5$ . [T]

## 5 Structure that governs the search

### 5.1 Products cannot amplify

**Theorem 6.** *Concatenating towers  $(d_1, \dots, d_k)$  and  $(e_1, \dots, e_l)$  gives  $n = n_1 n_2$  and  $m = m_1 n_2 + n_1 m_2$ , so the edge density  $m/n$  adds. Since the score is at least  $m/(n - |T|)$ , iterating a construction never improves its score. [T]*

Verified on Katz–Tao figure 2 squared:  $n = 36$ ,  $m = 84$  exactly as predicted, density  $7/3$  before a single generator is counted (Figure 3). [V] This is why the  $\gamma$  construction needs  $|X| \rightarrow \infty$ . Amortisation cannot come from repeating the geometry, because repetition costs edges faster than it buys vertices; it can only come from  $|R|$  staying bounded while  $n$  grows, and the only knob that does that is adding dilates. *More directions, not bigger graphs.* It is also Besicovitch’s economy: the directions proliferate while the material stays thin.

### 5.2 Components

Forcing decomposes over connected components: a witness supported on  $T \cup \{e\}$  can use only generators of  $e$ ’s component, so the module splits and the minimum generator count is the sum over components. Tested on 3513 disconnected instances with zero disagreements. [V]

### 5.3 A symmetry group at fixed score

**Proposition 7.**  *$M \in \text{GL}_2(\mathbb{Z})$  with  $(1, -1)M = \pm(1, -1)$  is exactly  $M = \begin{bmatrix} a & b \\ a-1 & b+1 \end{bmatrix}$  with  $a+b = \pm 1$  — two one-parameter integer families — and relabelling every label  $\mathbf{x} \mapsto \mathbf{x}M$  carries a forcing pair to a forcing pair of identical  $m, n, |R|, |T|$ . [T]*

Every one of twelve members tested carries figure 2 to a completing object at  $11/6$  on a different dilate set (Figure 3, right). [V] A dilate at level  $k = x_1 + x_2$  moves by multiples of  $k$  along its level line, so any level-1 dilate can be normalised to  $(1, 0)$ ; a pool search that does not quotient by this re-searches one orbit forever. This is the integral shadow of the affine invariance  $\beta(\phi(R)) = \beta(R)$  noted by Pohoata and Zakharov [9].

### 5.4 The offset is positional

On a fixed tower with fixed edge count, cyclic rearrangements of one label sequence — the same multiset by construction — give minimum generator counts differing by up to a factor of two: with  $T_0 = \{0\}$ , pool  $\{(1, 0), (0, 1), (1, 1), (1, 2)\}$  and  $n = 6$ ,  $d = (2, 3)$  varies on 9 of 14 sequences and  $d = (3, 2)$  on 7 of 14, with values 2, 3 and 4. It is not a special-slot effect: the same label on the level-1 slab gives 2, 3 and 4 in different rotations. The arrangement does the work. [V] (Measured in the session of 31 August; scripts AK-PHASE-01 to -04 in the author’s files, not reprinted here.) Together with Lemma 4 this says the difference information carried by edges and the absolute information carried by generators are not interchangeable quantities — the number of absolute values needed depends on where the differences sit.

## 6 The entropy route

Ruzsa’s equivalence makes the sums–differences problem an entropy inequality, and upper bounds of that shape are Shannon-type inequalities decidable by a linear program over the elemental inequalities. Two controls pass: the bare four projections return exactly 2, and the LP certifies Ruzsa’s entropy triangle inequality at 0.000000000. With two independent copies and cross differences — 255 unknowns, 1803 inequalities — the best provable constant is 2.000000. Not  $11/6$ ; not below. The content of  $11/6$  is the Balog–Szemerédi–Gowers step, which is not Shannon-type, and more copies cannot help. [V] The route is closed for a structural reason.

## 7 Engines and controls

Three forcing engines were built and cross-validated. The exact integer engine keeps the module in Hermite normal form and decides membership rather than enumerating. An independent rational engine, sharing no code, poses the support condition and the  $(1,1)$ -functional condition as one homogeneous system and asks whether the first coordinate is identically zero on its nullspace; a rational witness scales to an integer one, so the two must agree everywhere. On 6000 random instances across nine tower shapes and three dilate pools, 1150 completing, they agree on every one. A naive integer brute force with no linear algebra agrees on 150 tiny instances; its early mismatches were its own coefficient box being too small, witnesses needing coefficients up to  $\pm 6$  even on two vertices. [V]

Calibration on the published objects:

| object                      | tower        | $n$ | $m$ | $ R $ | result                      |
|-----------------------------|--------------|-----|-----|-------|-----------------------------|
| Katz–Tao figure 2           | $d = (2, 3)$ | 6   | 7   | 4     | forces $6/6$ , score $11/6$ |
| Katz–Tao figure 3           | $d = (2, 2)$ | 4   | 4   | 3     | forces $4/4$ , score $7/4$  |
| figure 2 less one generator |              | 6   | 7   | 3     | forces $0/6$ , score $5/3$  |
| figure 3 less one generator |              | 4   | 4   | 2     | forces $0/4$ , score $3/2$  |

The last two rows are the point: removing one generator sends the score below the record while the object forces nothing. Score is free; completeness is the content.

The control that makes every empty result mean something: with the budget raised by one, the search recovers an  $11/6$  object unaided — figure 2 up to column reflection. And the seeded enumerator of Sec. 8, given figure 2’s own labelling and  $|R| = 4$ , found in 519 calls an  $11/6$  object that is *not* Katz–Tao’s: generators  $(-1, 2)$  and  $(0, 1)$  at one vertex,  $(1, 2)$  and  $(2, -1)$  elsewhere. Richer dilates change which generators force even where they do not change how many. [V]

## 8 Searches

### 8.1 Closed regions

Goal 2, three-slope set,  $n - |T| = 5$ ,  $m + |R| \leq 9$ : nothing completes on  $d = (5)$  with  $t = 0$  (63,180 calls) or on  $d = (6)$ ,  $d = (2, 3)$ ,  $d = (3, 2)$  with  $t = 1$  (671,280, 346,392, 344,232). The smallest admissible size is empty across every tower with  $n \leq 6$ .

Goal 1, budget  $m + |R| \leq \lfloor 5(n - |T|)/3 \rfloor$ : nothing completes at  $n = 3, 4, 5$  (five-dilate pool), nor at  $n = 6$  with  $T$  empty over the Katz–Tao dilate set  $\{(1, 0), (0, 1), (1, 1), (1, 2)\}$  on  $d = (2, 3)$  and  $d = (3, 2)$  for every  $m$  from 0 to 7, nor on the 6-path at  $m = 5$  and  $m = 4$  and two-fifths of

$m = 3$ . The  $n = 4$  rows are a second free control: budget 6 on four free vertices is score  $3/2$ , the proven floor of the method, and nothing reaches it. [V]

## 8.2 The record hunt

Theorem 6 says which cell can pay: small  $n$ , richer dilates, fewer generators. The sharpest is the Katz–Tao ladder with one generator removed —  $n = 6$ , all seven edges,  $T$  empty,  $|R| = 3$  — which scores exactly  $10/6 = 5/3 \leq 1.675$ . A single completing object there is a new record.

Lemma 4 makes it finite:  $R$  is two independent generators at one vertex together with one generator anywhere. The rung is fixed at  $(1, 0)$ , which is without loss of generality for level-1 rungs by Proposition 7. Rails and generators range over the seven primitive dilates of height at most 2:  $(-1, 2), (0, 1), (1, 0), (1, 1), (1, 2), (2, -1), (2, 1)$ . Each of 49 sub-chunks is 49 rail labellings against 5040 seeded generator sets.

**Status** (Figure 4): 33 of 49 sub-chunks of  $d = (2, 3)$  closed, about 8.1 million verifier calls, nothing completes. Every sub-chunk runs the identical 246,960 calls, which is the signature of a pairwise-independent dilate set — the seeded enumerator sees the same combinatorial problem whatever labels sit where. The remaining 16 sub-chunks of  $d = (2, 3)$  and all 49 of  $d = (3, 2)$  are resumable from Appendix A.6. [V]

## 9 What is open

1. Finish the hunt at height 2 on both orientations. If both close empty the result is *no constructible proof at score  $5/3$  exists on the six-vertex ladder with a level-1 rung and dilates of height at most 2*, and the next axis is height 3, about 24 dilates, where the seeding lemma stops being a convenience and becomes the only thing that makes the search finite.
2. Rungs at levels other than 1; the  $d = (6)$  sparse strata;  $n = 7, 8$  with the component decomposition built in.
3. Katz 2006, the one prior-art hole. His IPAM talk on the sums–differences approach is free and is his own account of the barrier.
4. Whether the two formulations achieve the same scores.
5. What the arrangement dependence of Sec. 5.4 scales like, and whether it has a closed form.

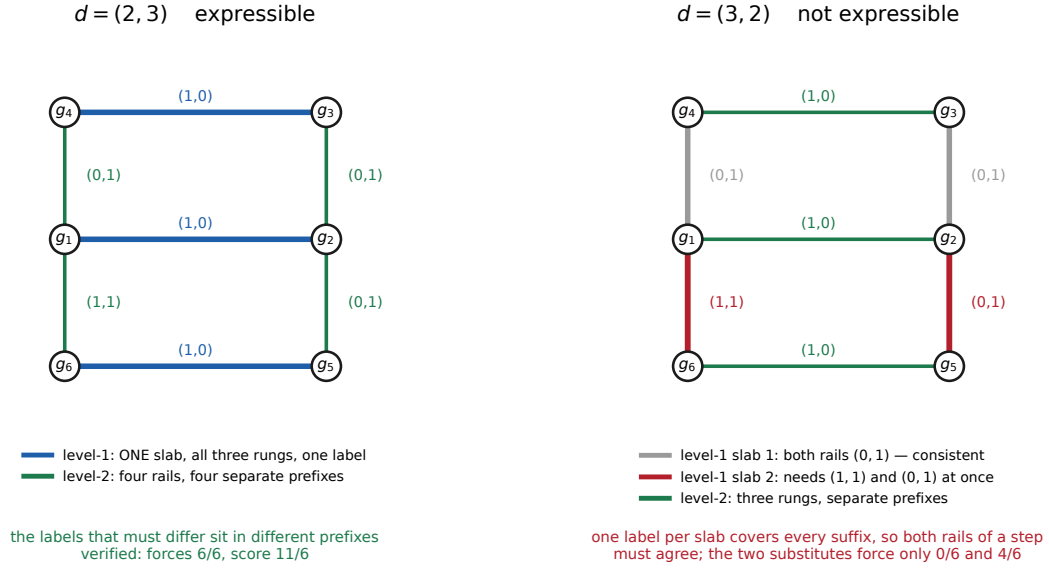


Figure 1: Katz–Tao’s six-vertex configuration in both tower orientations. Left,  $d = (2, 3)$ : the three rungs form one slab with the single label  $(1, 0)$  and the four rails sit in separate prefixes. Right,  $d = (3, 2)$ : the rails are the slabs, and the lower slab would need  $(1, 1)$  and  $(0, 1)$  under one label; the configuration is inexpressible there.

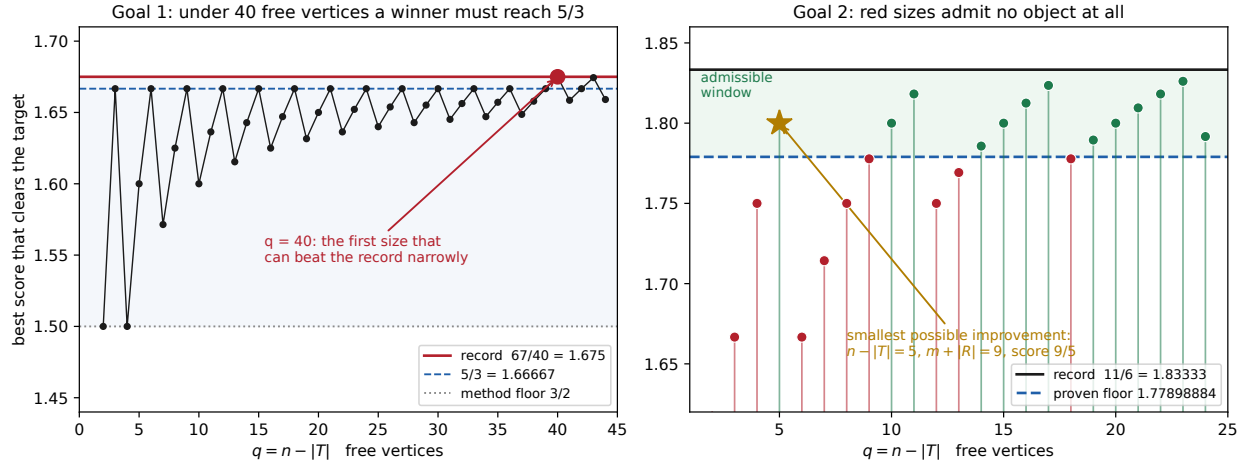


Figure 2: Score granularity. Left: below 40 free vertices the best score not exceeding the record is  $5/3$ . Right: for the sums–differences set, red sizes admit no object at all; the smallest possible improvement is  $q = 5$  with  $m + |R| = 9$ .



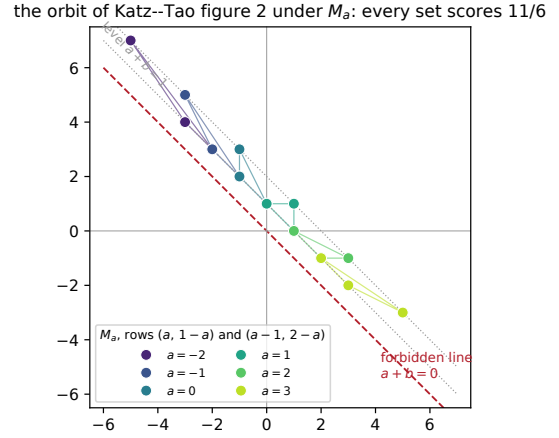
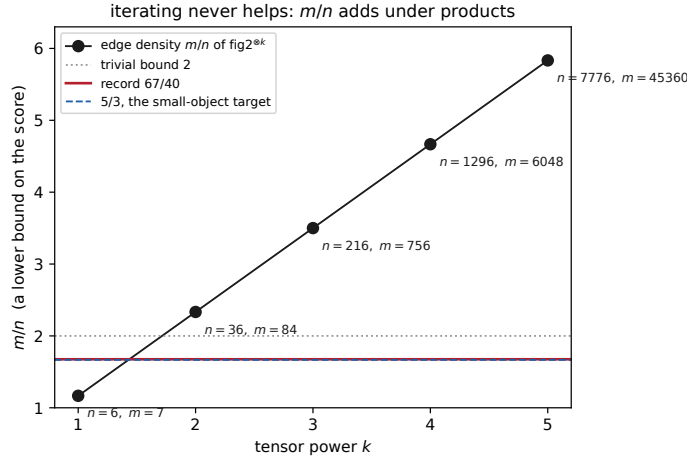


Figure 3: Left: edge density of figure 2 under repeated self-product. It adds, so the score of any iterate exceeds the trivial bound before generators are counted. Right: the dilate sets in the orbit of figure 2 under the symmetry  $M_a$ ; every one carries a completing object at exactly  $11/6$ . Dilates move along their level lines, never across the forbidden line.

The record hunt: rung  $(1, 0)$ ,  $m = 7$ ,  $|R| = 3$ , height-2 dilates; each cell is 49 rail labellings  $\times$  5040 seeded generator sets

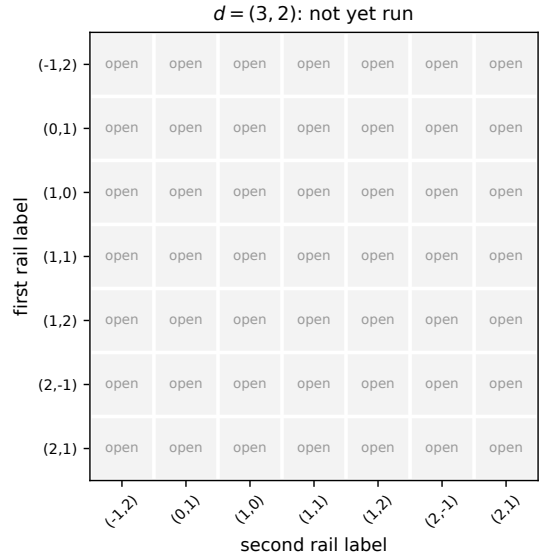


Figure 4: Coverage of the record hunt at height 2. Each cell is one sub-chunk; green cells are closed with nothing completing.

## A Scripts

All numbers in this note are produced by the scripts below, which run as printed.

### A.1 AK-KT-CALIB-01 — calibration against the published objects

```
1 # SCRIPT: AK-KT-CALIB-01
2 # -----
3 # WHAT THIS TESTS, AND WHY IT WOULD PROVE IT (written before the run)
4 #
5 # Everything downstream rests on two unexamined assumptions:
6 # (A1) the tower encoding (d_1..d_k + f_i on prefixes) can EXPRESS the objects
7 # Definition 1 of the Epoch write-up describes, and
8 # (A2) AK-VERIFIER-02 certifies a KNOWN-GOOD object at its published score.
9 # Neither has ever been tested. The verifier's only controls are the trivial
10 # AK(2) object and a two-vertex path; both score exactly 2, so they exercise no
11 # amortization and no nontrivial forcing cascade.
12 #
13 # The Epoch PDF supplies exactly the missing controls: Katz-Tao Figure 2
14 # (n=6, m=7, |R|=4, T=empty, score 11/6) and Figure 3 (n=4, m=4, |R|=3,
15 # T=empty, score 7/4). Both are published, both are correct.
16 #
17 # KILL CONDITION, fixed before running:
18 # * if either object comes back complete=False, then either the verifier is
19 # wrong or the tower encoding cannot hold the known record configurations --
20 # and every "nothing below 2" result from the shallow tower searches is
21 # uninterpretable, because the search space provably excludes the objects
22 # that already beat 2.
23 # * if m or n disagree with the published (6,7) and (4,4), the cost model
24 # m = sum_i sum_prefix [f_i != 0] * d_{i+1}...d_k is not Definition 1's
25 # edge count.
26 # A pass means: the encoding holds both published objects, the verifier finds
27 # the cascade unaided, and the scores come out exactly 11/6 and 7/4.
28 #
29 # THIRD TEST -- slab uniformity. In the tower, one label covers a whole slab.
30 # Figure 2 has two parallel edges (g1-g6 labelled (1,1), g2-g5 labelled (0,1))
31 # that carry DIFFERENT labels, so it is expressible only in the orientation
32 # where those two edges sit in different slabs. Test both orientations. If the
33 # wrong orientation fails, the tower encoding is orientation-dependent and any
34 # search must try every factor ordering, not just one.
35 # -----
36
37 import itertools
38 from fractions import Fraction
39 from math import gcd
40
41 def hnf(rows):
42     M = [list(r) for r in rows if any(r)]
43     if not M: return []
44     nc = len(M[0]); pr = 0
45     for c in range(nc):
46         while True:
47             nz = [r for r in range(pr, len(M)) if M[r][c] != 0]
48             if not nz: break
49             if len(nz) == 1:
50                 M[pr], M[nz[0]] = M[nz[0]], M[pr]; pr += 1; break
51             nz.sort(key=lambda r: abs(M[r][c])); p = nz[0]
52             for r in nz[1:]:
53                 q = M[r][c] // M[p][c]
54                 for j in range(nc): M[r][j] -= q*M[p][j]
55             if pr >= len(M): break
56     return sorted([r for r in M if any(r)])
57
58 def kernel_of_functional(rows, phi):
59     vals = [phi(r) for r in rows]
60     if all(v == 0 for v in vals): return [list(r) for r in rows]
```

```

61 out = []
62 piv = next(i for i,v in enumerate(vals) if v != 0)
63 for i,v in enumerate(vals):
64     if i == piv: continue
65     g = gcd(abs(v), abs(vals[piv])) or 1
66     a, b = vals[piv]//g, v//g
67     out.append([a*rows[i][t] - b*rows[piv][t] for t in range(len(rows[i]))])
68 return hnf(out)
69
70 def restrict_zero(rows, cols):
71     M = [list(r) for r in rows]
72     for c in cols:
73         if not M: return []
74         nz = [r for r in range(len(M)) if M[r][c] != 0]
75         while len(nz) > 1:
76             nz.sort(key=lambda r: abs(M[r][c])); p = nz[0]
77             for r in nz[1:]:
78                 q = M[r][c] // M[p][c]
79                 for j in range(len(M[r])): M[r][j] -= q*M[p][j]
80             nz = [r for r in range(len(M)) if M[r][c] != 0]
81         if nz: M = [M[r] for r in range(len(M)) if r != nz[0]]
82         M = [r for r in M if any(r)]
83     return hnf(M)
84
85 class AK:
86     def __init__(self, X, dims, fs, T, R):
87         self.X = [tuple(x) for x in X]
88         assert (0,0) in self.X
89         for (a,b) in self.X:
90             if (a,b) != (0,0): assert a+b != 0, "X element with a+b=0: %s" % ((a,b),)
91         self.d = list(dims); self.k = len(dims); self.fs = fs
92         self.V = [tuple(e) for e in itertools.product(*[range(1,d+1) for d in self.d])]
93         self.idx = {v:i for i,v in enumerate(self.V)}
94         self.T0 = set(map(tuple, T))
95         self.R0 = [{tuple(v): tuple(x) for v,x in r.items()} for r in R]
96         for r in self.R0:
97             assert len([1 for v,x in r.items() if tuple(x) != (0,0)]) == 1
98         for i in range(self.k):
99             for pref,x in self.fs[i].items():
100                 assert tuple(x) in self.X, "label %s not in X" % (x,)
101                 assert len(pref) == i+1, "prefix %s wrong length for level %d" % (pref,i+1)
102                 assert 1 <= pref[-1] <= self.d[i]-1, "prefix %s out of range" % (pref,)
103     def n(self):
104         p=1
105         for d in self.d: p*=d
106         return p
107     def m(self):
108         tot=0
109         for i in range(self.k):
110             tail=1
111             for j in range(i+1,self.k): tail*=self.d[j]
112             for pref,x in self.fs[i].items():
113                 if tuple(x) != (0,0): tot+=tail
114         return tot
115     def vec(self, fd):
116         v=[0]*(2*len(self.V))
117         for w,x in fd.items():
118             i=self.idx[tuple(w)]; v[2*i]+=x[0]; v[2*i+1]+=x[1]
119         return v
120     def edge_gens(self):
121         out=[]
122         for i in range(self.k):
123             for pref,x in self.fs[i].items():
124                 x=tuple(x)
125                 if x==(0,0): continue
126                 a=list(pref)
127                 for tail in itertools.product(*[range(1,self.d[j]+1) for j in range(i+1,self.k)]):
128                     e1=tuple(a)+tail; e2=tuple(a[:-1]+[a[-1]+1])+tail

```

```

129         if e1 in self.idx and e2 in self.idx:
130             out.append({e1:x, e2:(-x[0],-x[1])})
131     return out
132 def edge_list(self):
133     return [(sorted(g.keys())[0], sorted(g.keys())[1],
134             g[[v for v in g if g[v][0]>=0 or g[v][1]>=0][0]]) for g in self.edge_gens()]
135 def run(self):
136     B = hnf([self.vec(r) for r in self.R0] + [self.vec(g) for g in self.edge_gens()])
137     T = set(self.T0); prog=True; order=[]
138     while prog and len(T) < len(self.V):
139         prog=False
140         for e in self.V:
141             if e in T: continue
142             free = T | {e}
143             cols=[]
144             for v in self.V:
145                 if v not in free: cols += [2*self.idx[v], 2*self.idx[v]+1]
146             S = restrict_zero(B, cols)
147             if not S: continue
148             ei = self.idx[e]
149             K = kernel_of_functional(S, lambda w: w[2*ei] + w[2*ei+1])
150             if any(w[2*ei] != 0 for w in K):
151                 T.add(e); order.append(e); prog=True
152     return T, order
153 def report(self, name):
154     T,order=self.run(); ok=(len(T)==len(self.V))
155     num=self.m()+len(self.R0); den=self.n()-len(self.T0)
156     s=Fraction(num,den)
157     print(" %-26s n=%-3d m=%-3d |R|=%-2d |T0|=%-2d forced %d/%d complete=%-5s score=%s = %.5f"
158           %(name,self.n(),self.m(),len(self.R0),len(self.T0),len(T),len(self.V),ok,s,float(s)))
159     return ok,s,order
160
161 # -----
162 # KATZ-TAO FIGURE 2 -- score 11/6, X = {(1,0),(0,1),(1,1)}
163 #
164 #   g4 --(1,0)-- g3           tower orientation d = (2,3):
165 #   |             |           e1 = column (1 = left, 2 = right)
166 # (0,1)         (0,1)         e2 = row    (1 = top, 2 = middle, 3 = bottom)
167 #   |             |
168 #   g1 --(1,0)-- g2           (1,1)=g4 (2,1)=g3
169 #   |             |           (1,2)=g1 (2,2)=g2
170 # (1,1)         (0,1)         (1,3)=g6 (2,3)=g5
171 #   |             |
172 #   g6 --(1,0)-- g5
173 #
174 # level 1 (one slab, label (1,0)) supplies all three rungs;
175 # level 2 supplies the four rails, each its own prefix, so each its own label.
176 # -----
177 X3 = [(0,0),(1,0),(0,1),(1,1)]
178 NAME2 = {(1,1):"g4",(2,1):"g3",(1,2):"g1",(2,2):"g2",(1,3):"g6",(2,3):"g5"}
179
180 fig2 = AK(X3, [2,3],
181           [ {(1,): (1,0)},
182             {(1,1): (0,1), (1,2): (1,1), (2,1): (0,1), (2,2): (0,1)} ],
183           [],
184           [ {(1,2): (1,0)},      # (1,0) g1
185             {(2,1): (1,1)},      # (1,1) g3
186             {(1,1): (1,1)},      # (1,1) g4
187             {(1,3): (0,1)} ])    # (0,1) g6
188
189 print("KATZ-TAO FIGURE 2 (published: n=6, m=7, |R|=4, T empty, score 11/6 = 1.83333)")
190 ok2, s2, order2 = fig2.report("KT fig2, d=(2,3)")
191 print(" published m,n = 7,6      computed m,n = ", fig2.m(), ", ", fig2.n(),
192       " match:", (fig2.m(), fig2.n()) == (7,6))
193 print(" published score 11/6      computed score", s2, " match:", s2 == Fraction(11,6))
194 print(" forcing order:", " -> ".join(NAME2[v] for v in order2))
195 print(" (Epoch's text forces g5 first, then g3, g4, g1, g2, g6.)")
196

```

```

197 # -----
198 # ORIENTATION CONTROL -- the same graph, factors swapped: d = (3,2).
199 # Now the two rails g1-g6 and g2-g5 lie in ONE slab and must share a label,
200 # but Figure 2 gives them (1,1) and (0,1). The figure is therefore NOT
201 # expressible in this orientation; the closest tower objects are the two
202 # choices below. Neither is Figure 2 -- this is a check on the encoding, not
203 # on Katz-Tao.
204 # -----
205 print("\nORIENTATION CONTROL -- same drawing, d=(3,2): rails g1-g6 and g2-g5 share one slab")
206 for lab in [(1,1),(0,1)]:
207     alt = AK(X3, [3,2],
208             [ {(1,):(0,1)}, {(2,):lab},
209               {(1,1):(1,0)}, {(2,1):(1,0)}, {(3,1):(1,0)} ],
210             [],
211             [ {(2,1):(1,0)}, {(1,2):(1,1)}, {(1,1):(1,1)}, {(3,1):(0,1)} ])
212     alt.report("d=(3,2), both rails %s" % (lab,))
213
214 # -----
215 # KATZ-TAO FIGURE 3 -- score 7/4, X = {(1,0),(0,1),(1,2),(1,1)}
216 #
217 #   g4 --(1,0)-- g3           tower orientation d = (2,2):
218 #   |             |           (1,1)=g4 (2,1)=g3
219 # (1,2)         (0,1)       (1,2)=g1 (2,2)=g2
220 #   |             |
221 #   g1 --(1,0)-- g2
222 # -----
223 X4 = [(0,0),(1,0),(0,1),(1,1),(1,2)]
224 NAME3 = {(1,1):"g4", (2,1):"g3", (1,2):"g1", (2,2):"g2"}
225
226 fig3 = AK(X4, [2,2],
227           [ {(1,):(1,0)},
228             {(1,1):(1,2)}, {(2,1):(0,1)} ],
229           [],
230           [ {(1,2):(1,1)}, # (1,1) g1
231             {(2,2):(1,1)}, # (1,1) g2
232             {(1,1):(0,1)} ]) # (0,1) g4
233
234 print("\nKATZ-TAO FIGURE 3 (published: n=4, m=4, |R|=3, T empty, score 7/4 = 1.75)")
235 ok3, s3, order3 = fig3.report("KT fig3, d=(2,2)")
236 print(" published m,n = 4,4      computed m,n =", fig3.m(), ",", fig3.n(),
237       " match:", (fig3.m(), fig3.n()) == (4,4))
238 print(" published score 7/4      computed score", s3, " match:", s3 == Fraction(7,4))
239 print(" forcing order:", " -> ".join(NAME3[v] for v in order3))
240 print(" (Epoch's text forces g3 first, then g4, g1, g2.)")
241
242 # -----
243 # NEGATIVE CONTROL -- drop one generator from each object. A correct verifier
244 # must then fail to complete; if it still completes, it is manufacturing
245 # forcing out of nothing and both passes above are worthless.
246 # -----
247 print("\nNEGATIVE CONTROLS -- remove one element of R; completion must FAIL")
248 AK(X3, [2,3],
249     [ {(1,):(1,0)}, {(1,1):(0,1)}, {(1,2):(1,1)}, {(2,1):(0,1)}, {(2,2):(0,1)} ],
250     [], [ {(1,2):(1,0)}, {(2,1):(1,1)}, {(1,1):(1,1)} ]).report("fig2 minus (0,1)g6")
251 AK(X4, [2,2],
252     [ {(1,):(1,0)}, {(1,1):(1,2)}, {(2,1):(0,1)} ],
253     [], [ {(1,2):(1,1)}, {(2,2):(1,1)} ]).report("fig3 minus (0,1)g4")
254
255 print("\nVERDICT")
256 print(" encoding holds both published objects :", ok2 and ok3)
257 print(" scores reproduce exactly                :", s2 == Fraction(11,6) and s3 == Fraction(7,4))

```

## A.2 AK-GRANULARITY-01 — which sizes can possibly beat each target

```

1 # SCRIPT: AK-GRANULARITY-01
2 # -----
3 # A score is  $(m + |R|) / (n - |T|)$ , a ratio of non-negative integers, so the two
4 # targets are not approached continuously. This computes, for each denominator
5 #  $q = n - |T|$ , the largest score that clears each target, and crosses it against
6 # the proven lower bound for the three-slope set.
7 #
8 #   GOAL 1   score  $\leq 67/40 = 1.675$           (any finite  $X$ )
9 #   GOAL 2   score  $< 11/6 = 1.83333\dots$     ( $X = \{(1,0), (0,1), (1,1)\}$ )
10 #           and, for that  $X$ , score  $\geq 1.77898884$  is forced by the proven lower
11 #           bound on the sum-difference constant, so any  $q$  whose best fraction
12 #           falls below that bound admits NO object at all.
13 # -----
14
15 from fractions import Fraction
16
17 G1   = Fraction(67, 40)
18 SD   = Fraction(11, 6)
19 FLOOR = 1.77898884          # proven lower bound, sum-difference constant
20
21 print("GOAL 1 -- largest score  $\leq 67/40$  available at each denominator  $q = n - |T|$ ")
22 rows = []
23 for q in range(1, 45):
24     p = (67*q)//40
25     rows.append((q, p, Fraction(p, q)))
26 for q, p, f in rows:
27     if q  $\leq 12$  or  $q \geq 36$ :
28         print("    q=%-3d max m+|R| = %-3d best score %-8s = %.5f" % (q, p, f, float(f)))
29 sub40 = max(f for q, p, f in rows if q < 40)
30 print("    ---> largest score below 67/40 available with  $q < 40$  :", sub40,
31       "= %.5f" % float(sub40))
32 print("    ---> so any object with fewer than 40 free vertices that beats the")
33 print("    record must beat it all the way down to 5/3, not narrowly.")
34
35 print()
36 print("GOAL 2 -- score strictly below 11/6, crossed against the proven floor")
37 adm = []
38 for q in range(1, 25):
39     p = 11*q//6
40     if Fraction(p, q)  $\geq$  SD: p -= 1
41     f = Fraction(p, q)
42     ok = float(f)  $\geq$  FLOOR
43     if ok: adm.append(q)
44     print("    n-|T| = %-3d best m+|R| = %-3d score %-7s = %.5f    %s"
45           % (q, p, f, float(f),
46              "admissible" if ok else "below the proven floor: no object exists"))
47 print("    ---> admissible sizes up to 24 :", adm)
48 print("    ---> smallest possible improvement:  $n-|T| = 5$  with  $m + |R| = 9$ ,")
49 print("    score  $9/5 = 1.8$ ")

```

### A.3 AK-COVER-SEARCH-01 — engine, obstruction, covering search, controls

```

1 # SCRIPT: AK-COVER-SEARCH-01
2 # -----
3 # Exhaustive search for constructible proofs of the arithmetic Kakeya bound,
4 # in the verifiable tower encoding of the Epoch FrontierMath open problem.
5 #
6 # CONTENTS
7 #   1. exact integer forcing engine (operations 1-3 over Z, no tolerances)
8 #   2. tower geometry: d_1..d_k plus slot labels -> vertices and edges
9 #   3. the SUBSET RANK OBSTRUCTION and the covering search built on it
10 #   4. controls, run first, every time
11 #
12 # THE OBSTRUCTION THE SEARCH IS BUILT ON
13 #   If vertex e falls, some module element w is supported on T u {e} with
14 #   w(e) = (a,-a), a nonzero. Fix any S contained in V\T with e in S and sum
15 #   w's columns over S. Each edge generator contributes +-x_E once if E crosses
16 #   the boundary of S and nothing at all otherwise (both endpoints inside cancel;
17 #   both outside contribute nothing). Each vertex generator contributes its own
18 #   label if it sits in S. Hence
19 #       w(e) in L(S) := span{ generators in S } + span{ labels crossing dS }.
20 #   Every label has s(x) = x1 + x2 nonzero, so a rank-1 lattice contains no
21 #   (a,-a) with a nonzero. Therefore rank L(S) = 2.
22 #   Now take ANY nonempty S contained in V\T0 and let e be the FIRST vertex of S
23 #   to fall: at that moment no vertex of S is known, so S is contained in the
24 #   unknown set and the argument applies. Hence
25 #
26 #       rank L(S) = 2   for EVERY nonempty S contained in V \ T0.
27 #
28 #   S = {e} gives a per-vertex condition; S = V\T0 gives the global one. The
29 #   search enumerates generator sets that satisfy every S by construction rather
30 #   than enumerating all generator sets and discarding those that fail.
31 #
32 # KILL CONDITIONS, fixed before any run
33 #   * the engine must verify Katz-Tao figure 2 at 11/6 and figure 3 at 7/4;
34 #   * with the budget raised by one the SEARCH must find an 11/6 object by
35 #     itself -- otherwise an empty result means nothing;
36 #   * on X = {(1,0),(0,1),(1,1)} no completing object may score below
37 #     1.77898884, the proven lower bound on the sum-difference constant; such an
38 #     object would mean the engine is broken, not that the record fell.
39 # -----
40
41 import itertools, time, sys
42 from fractions import Fraction
43 from math import gcd
44
45 # ----- 1. exact integer forcing engine -----
46
47 def hnf(rows):
48     M = [list(r) for r in rows if any(r)]
49     if not M: return []
50     nc = len(M[0]); pr = 0
51     for c in range(nc):
52         while True:
53             nz = [r for r in range(pr, len(M)) if M[r][c] != 0]
54             if not nz: break
55             if len(nz) == 1:
56                 M[pr], M[nz[0]] = M[nz[0]], M[pr]; pr += 1; break
57             nz.sort(key=lambda r: abs(M[r][c])); p = nz[0]
58             for r in nz[1:]:
59                 q = M[r][c] // M[p][c]
60                 for j in range(nc): M[r][j] -= q*M[p][j]
61             if pr >= len(M): break
62     return [r for r in M if any(r)]
63
64 def kernel_of_functional(rows, i2):
65     vals = [r[i2] + r[i2+1] for r in rows]
66     if all(v == 0 for v in vals): return [list(r) for r in rows]

```

```

67     out = []
68     piv = next(i for i, v in enumerate(vals) if v != 0)
69     for i, v in enumerate(vals):
70         if i == piv: continue
71         g = gcd(abs(v), abs(vals[piv])) or 1
72         a, b = vals[piv]//g, v//g
73         out.append([a*rows[i][t] - b*rows[piv][t] for t in range(len(rows[i]))])
74     return hnf(out)
75
76 def restrict_zero(rows, cols):
77     M = [list(r) for r in rows]
78     for c in cols:
79         if not M: return []
80         nz = [r for r in range(len(M)) if M[r][c] != 0]
81         while len(nz) > 1:
82             nz.sort(key=lambda r: abs(M[r][c])); p = nz[0]
83             for r in nz[1:]:
84                 q = M[r][c] // M[p][c]
85                 for j in range(len(M[r])): M[r][j] -= q*M[p][j]
86             nz = [r for r in range(len(M)) if M[r][c] != 0]
87         if nz: M = [M[r] for r in range(len(M)) if r != nz[0]]
88         M = [r for r in M if any(r)]
89     return hnf(M)
90
91 def forces(nv, edges, gens, T0):
92     """edges: (i, j, x).  gens: (v, x).  T0: set of vertex indices already known.
93     Operation 3 is handled by keeping the module in Hermite normal form, so
94     Z-membership is exact and nothing is enumerated."""
95     rows = []
96     for (i, j, x) in edges:
97         v = [0]*(2*nv)
98         v[2*i] += x[0]; v[2*i+1] += x[1]; v[2*j] -= x[0]; v[2*j+1] -= x[1]
99         rows.append(v)
100     for (g, x) in gens:
101         v = [0]*(2*nv); v[2*g] += x[0]; v[2*g+1] += x[1]
102         rows.append(v)
103     B = hnf(rows); T = set(T0); prog = True
104     while prog and len(T) < nv:
105         prog = False
106         for e in range(nv):
107             if e in T: continue
108             cols = [c for v in range(nv) if v != e and v not in T for c in (2*v, 2*v+1)]
109             S = restrict_zero(B, cols)
110             if not S: continue
111             K = kernel_of_functional(S, 2*e)
112             if any(w[2*e] != 0 for w in K):
113                 T.add(e); prog = True
114     return len(T) == nv
115
116 # ----- 2. tower geometry -----
117
118 def tower(dims):
119     V = [tuple(e) for e in itertools.product(*[range(1, d+1) for d in dims])]
120     idx = {v: i for i, v in enumerate(V)}
121     slots = []
122     for i in range(len(dims)):
123         pref = list(itertools.product(*[range(1, dims[j]+1) for j in range(i)],
124                                     range(1, dims[i])))
125         tail = 1
126         for j in range(i+1, len(dims)): tail *= dims[j]
127         slots.append((i, pref, tail))
128     return V, idx, slots
129
130 def build_edges(dims, V, idx, slots, labels):
131     """One label per prefix creates a whole slab of edges and costs the whole slab."""
132     edges = []; m = 0
133     for (i, pref, tail) in slots:
134         for p in pref:

```



```

135         x = labels[(i, p)]
136         if x == (0, 0): continue
137         m += tail
138         a = list(p)
139         for t in itertools.product(*[range(1, dims[j]+1)
140                                     for j in range(i+1, len(dims))]):
141             e1 = tuple(a)+t; e2 = tuple(a[:-1]+[a[-1]+1])+t
142             edges.append((idx[e1], idx[e2], x))
143     return edges, m
144
145 def rank_of(vs):
146     b = []
147     for v in vs:
148         if v == (0, 0): continue
149         if not b: b.append(v)
150         elif b[0][0]*v[1] - b[0][1]*v[0] != 0: return 2
151     return len(b)
152
153 # ----- 3. covering search -----
154
155 def search(dims, POOL, t, budget, mmin=0, mmax=10**9, tlimit=None):
156     V, idx, slots = tower(dims); nv = len(V)
157     slotlist = [(i, p) for (i, pref, tail) in slots for p in pref]
158     vals = [(0, 0)] + POOL
159     best = [None]; bestobj = [None]; calls = [0]; nodes = [0]
160     t0 = time.time(); done = [True]
161     for combo in itertools.product(vals, repeat=len(slotlist)):
162         if not done[0]: break
163         if tlimit and time.time()-t0 > tlimit: done[0] = False; break
164         labels = dict(zip(slotlist, combo))
165         edges, m = build_edges(dims, V, idx, slots, labels)
166         if m > budget or m < mmin or m > mmax: continue
167         inc = [[] for _ in range(nv)]
168         for (i, j, x) in edges: inc[i].append(x); inc[j].append(x)
169         for T0 in itertools.combinations(range(nv), t):
170             T0s = set(T0); free = [v for v in range(nv) if v not in T0s]
171             defsub = []
172             for k in range(1, len(free)+1):
173                 for S in itertools.combinations(free, k):
174                     Ss = set(S)
175                     vs = [x for (i, j, x) in edges if (i in Ss) != (j in Ss)]
176                     if rank_of(vs) < 2:
177                         nz = [x for x in vs if x != (0, 0)]
178                         defsub.append((Ss, nz[0] if nz else None))
179             need = [max(0, 2 - rank_of(inc[v])) for v in range(nv)]
180             if m + sum(need[v] for v in free) > budget: continue
181             rmax = budget - m
182             allgens = [(v, x) for v in free for x in POOL]
183             ng = len(allgens)
184             elig = [set(gi for gi, (v, x) in enumerate(allgens) if v in Ss)
185                     for (Ss, d0) in defsub]
186
187             def satisfied(chosen, si):
188                 Ss, d0 = defsub[si]
189                 vs = [allgens[gi][1] for gi in chosen if allgens[gi][0] in Ss]
190                 if d0 is not None: vs = vs + [d0]
191                 return rank_of(vs) >= 2
192
193             def dfs(i, chosen):
194                 nodes[0] += 1
195                 if nodes[0] % 4096 == 0 and tlimit and time.time()-t0 > tlimit:
196                     done[0] = False
197                 if not done[0]: return
198                 unsat = [si for si in range(len(defsub)) if not satisfied(chosen, si)]
199                 if not unsat:
200                     s = Fraction(m + len(chosen), nv - t)
201                     if best[0] is None or s < best[0]:
202                         calls[0] += 1

```

```

203         if forces(nv, edges, [allgens[g] for g in chosen], T0s):
204             best[0] = s
205             bestobj[0] = (dims,
206                           {k: v for k, v in labels.items() if v != (0, 0)},
207                           [V[q] for q in T0],
208                           [(V[allgens[g][0]], allgens[g][1]) for g in chosen],
209                           m, len(chosen))
210             return
211         if len(chosen) >= rmax: return
212         for si in unsat:
213             if not any(gi >= i for gi in elig[si]): return
214         bound = 0; used = set()
215         for si in unsat:
216             Ss = defsub[si][0]
217             if not (Ss & used): bound += 1; used |= Ss
218         if len(chosen) + bound > rmax: return
219         for gi in range(i, ng):
220             dfs(gi+1, chosen + [gi])
221
222     dfs(0, [])
223     return best[0], bestobj[0], calls[0], done[0]
224
225 # ----- 4. controls -----
226
227 SD3 = [(1, 0), (0, 1), (1, 1)]           # the sums-differences set
228 KTX = [(1, 0), (0, 1), (1, 1), (1, 2)]  # both Katz-Tao configurations
229
230 def kt_figures():
231     V, idx, slots = tower([2, 3])
232     lab = {(0, (1,)): (1, 0), (1, (1, 1)): (0, 1), (1, (1, 2)): (1, 1),
233            (1, (2, 1)): (0, 1), (1, (2, 2)): (0, 1)}
234     e, m = build_edges([2, 3], V, idx, slots, lab)
235     g = [(idx[(1, 2)], (1, 0)), (idx[(2, 1)], (1, 1)),
236          (idx[(1, 1)], (1, 1)), (idx[(1, 3)], (0, 1))]
237     print(" KT figure 2  d=(2,3) n=6 m=%d r=%d complete=%s score=%s"
238           % (m, len(g), forces(6, e, g, set()), Fraction(m+len(g), 6)))
239     V, idx, slots = tower([2, 2])
240     lab = {(0, (1,)): (1, 0), (1, (1, 1)): (1, 2), (1, (2, 1)): (0, 1)}
241     e, m = build_edges([2, 2], V, idx, slots, lab)
242     g = [(idx[(1, 2)], (1, 1)), (idx[(2, 2)], (1, 1)), (idx[(1, 1)], (0, 1))]
243     print(" KT figure 3  d=(2,2) n=4 m=%d r=%d complete=%s score=%s"
244           % (m, len(g), forces(4, e, g, set()), Fraction(m+len(g), 4)))
245     # the same drawing in the reversed tower orientation cannot carry two
246     # different labels on the two rails of one slab
247     for lab2 in [(1, 1), (0, 1)]:
248         V, idx, slots = tower([3, 2])
249         lab = {(0, (1,)): (0, 1), (0, (2,)): lab2,
250                (1, (1, 1)): (1, 0), (1, (2, 1)): (1, 0), (1, (3, 1)): (1, 0)}
251         e, m = build_edges([3, 2], V, idx, slots, lab)
252         g = [(idx[(2, 1)], (1, 0)), (idx[(1, 2)], (1, 1)),
253              (idx[(1, 1)], (1, 1)), (idx[(3, 1)], (0, 1))]
254         print(" orientation control d=(3,2), both rails %s: complete=%s"
255               % (lab2, forces(6, e, g, set())))
256
257 if __name__ == "__main__":
258     print("CONTROLS")
259     kt_figures()
260     print("\nPOSITIVE CONTROL -- the SEARCH must find an 11/6 object by itself")
261     t0 = time.time()
262     b, o, c, dn = search([2, 3], SD3, 0, 11, 7, 7)
263     print(" d=(2,3) t=0 budget 11, m=7 : best=%s (%d verifier calls, %.1fs)"
264           % (b, c, time.time()-t0))
265     print(" object:", o)
266     print("\nNEGATIVE CONTROL -- smallest admissible size for beating 11/6")
267     t0 = time.time()
268     b, o, c, dn = search([5], SD3, 0, 9)
269     print(" d=(5) t=0 budget 9 : best=%s (%d verifier calls, %.1fs)"

```

270

```
% (b, c, time.time()-t0))
```

## A.4 AK-ENGINE-CROSSCHECK-01 — three independent engines

```

1 # SCRIPT: AK-ENGINE-CROSSCHECK-01
2 # -----
3 # WHY THIS EXISTS, AND WHAT IT WOULD PROVE
4 #
5 # Every "nothing completes" result rests on the custom integer lattice routines
6 # in AK-COVER-SEARCH-01 (hnf, restrict_zero, kernel_of_functional). Verifying
7 # those against the two published Katz-Tao objects tests only ONE direction:
8 # that the engine does not invent forcing where there is none. The dangerous
9 # direction is the opposite one. If a valid integer combination exists but the
10 # reduction fails to expose it, the engine reports "not forced" for an object
11 # that IS forced, the search silently discards good candidates, and every empty
12 # region in the write-up is worthless.
13 #
14 # THE EQUIVALENCE THAT MAKES A SECOND ENGINE LEGITIMATE
15 # Operation 2 asks for a module element w, supported on T u {e}, with
16 # w(e) = (a,-a) and a nonzero. Over Q this is the same question as over Z: if a
17 # rational combination gives such a w, multiplying by the common denominator of
18 # its coefficients gives an integer combination with w(e) = (Da,-Da), still
19 # nonzero and still supported on T u {e}. Conversely an integer witness is a
20 # rational one. So a rational engine must agree with the integer engine on
21 # EVERY instance, and any disagreement is a bug in one of them.
22 #
23 # THE SECOND ENGINE SHARES NO CODE WITH THE FIRST. It uses sympy's exact
24 # rational nullspace, and it poses the question differently: instead of
25 # reducing the module and then testing a functional, it writes the support
26 # condition and the (1,1)-functional condition as ONE homogeneous system in the
27 # generator coefficients, takes its nullspace, and asks whether the map
28 # c -> (cM)(e) first coordinate is identically zero there.
29 #
30 # A THIRD, NAIVE ENGINE is included for small instances: brute force over
31 # integer coefficient vectors in a small box, looking for an explicit witness.
32 # It is far too slow to search with, but it needs no linear algebra at all, so
33 # it is the control on the two controls.
34 #
35 # KILL CONDITION, fixed before the run:
36 # ANY disagreement between the integer and rational engines is a defect.
37 # In particular a single case with integer=NOT FORCED and rational=FORCED
38 # invalidates every empty search region reported so far.
39 # -----
40
41 import itertools, random, time
42 from fractions import Fraction
43 import sympy
44
45 exec(open('AK-COVER-SEARCH-01.py').read().split('# ----- 4. controls')[0])
46 # the above supplies: hnf, kernel_of_functional, restrict_zero, forces (integer
47 # engine), tower, build_edges, rank_of
48
49 # ----- engine B: exact rational, independent implementation -----
50
51 def forces_Q(nv, edges, gens, T0):
52     """Same question, rational nullspace formulation, no shared code."""
53     M = []
54     for (i, j, x) in edges:
55         row = [0]*(2*nv)
56         row[2*i] += x[0]; row[2*i+1] += x[1]
57         row[2*j] -= x[0]; row[2*j+1] -= x[1]
58         M.append(row)
59     for (g, x) in gens:
60         row = [0]*(2*nv); row[2*g] += x[0]; row[2*g+1] += x[1]
61         M.append(row)
62     if not M: return len(T0) == nv
63     Msym = sympy.Matrix(M) # rows = generators
64     T = set(T0); progress = True
65     while progress and len(T) < nv:
66         progress = False

```

```

67     for e in range(nv):
68         if e in T: continue
69         forbidden = [c for v in range(nv) if v != e and v not in T
70                     for c in (2*v, 2*v+1)]
71         # one homogeneous system: support condition, then the (1,1) condition
72         cons = [Msym[:, c].T for c in forbidden]
73         cons.append((Msym[:, 2*e] + Msym[:, 2*e+1]).T)
74         A = sympy.Matrix.vstack(*cons) if cons else sympy.zeros(1, Msym.rows)
75         N = A.nullspace() # coefficient vectors c
76         if not N: continue
77         # is c -> (cM)(e)_1 identically zero on the nullspace?
78         col = Msym[:, 2*e]
79         if any((sympy.Matrix(c).T * col)[0, 0] != 0 for c in N):
80             T.add(e); progress = True
81     return len(T) == nv
82
83 # ----- engine C: naive integer brute force, tiny instances -----
84
85 def forces_brute(nv, edges, gens, T0, box=2):
86     """No linear algebra at all: search integer coefficient vectors directly."""
87     rows = []
88     for (i, j, x) in edges:
89         r = [0]*(2*nv)
90         r[2*i] += x[0]; r[2*i+1] += x[1]; r[2*j] -= x[0]; r[2*j+1] -= x[1]
91         rows.append(r)
92     for (g, x) in gens:
93         r = [0]*(2*nv); r[2*g] += x[0]; r[2*g+1] += x[1]
94         rows.append(r)
95     if len(rows) > 5: return None # refuse: too big to brute force
96     T = set(T0); progress = True
97     while progress and len(T) < nv:
98         progress = False
99         for e in range(nv):
100             if e in T: continue
101             ok = False
102             for c in itertools.product(range(-box, box+1), repeat=len(rows)):
103                 if not any(c): continue
104                 w = [sum(c[k]*rows[k][t] for k in range(len(rows)))
105                     for t in range(2*nv)]
106                 if any(w[2*v] or w[2*v+1] for v in range(nv)
107                     if v != e and v not in T): continue
108                 if w[2*e] + w[2*e+1] == 0 and w[2*e] != 0:
109                     ok = True; break
110             if ok: T.add(e); progress = True
111     return len(T) == nv
112
113 # ----- controls -----
114
115 def kt2():
116     V, idx, slots = tower([2, 3])
117     lab = {(0, (1,)): (1, 0), (1, (1, 1)): (0, 1), (1, (1, 2)): (1, 1),
118           (1, (2, 1)): (0, 1), (1, (2, 2)): (0, 1)}
119     e, m = build_edges([2, 3], V, idx, slots, lab)
120     g = [(idx[(1, 2)], (1, 0)), (idx[(2, 1)], (1, 1)),
121          (idx[(1, 1)], (1, 1)), (idx[(1, 3)], (0, 1))]
122     return 6, e, g
123
124 def kt3():
125     V, idx, slots = tower([2, 2])
126     lab = {(0, (1,)): (1, 0), (1, (1, 1)): (1, 2), (1, (2, 1)): (0, 1)}
127     e, m = build_edges([2, 2], V, idx, slots, lab)
128     g = [(idx[(1, 2)], (1, 1)), (idx[(2, 2)], (1, 1)), (idx[(1, 1)], (0, 1))]
129     return 4, e, g
130
131 if __name__ == "__main__":
132     print("CONTROLS -- both engines on the published objects")
133     for name, (nv, e, g) in [("KT figure 2", kt2()), ("KT figure 3", kt3())]:
134         print(" %-12s integer=%-5s rational=%-5s"

```

```

135         % (name, forces(nv, e, g, set()), forces_Q(nv, e, g, set()))
136 for name, (nv, e, g) in [("fig2 less one", kt2()), ("fig3 less one", kt3())]:
137     print(" %-12s integer=%-5s rational=%-5s"
138           % (name, forces(nv, e, g[:-1], set()), forces_Q(nv, e, g[:-1], set())))
139
140 print("\nCROSS-VALIDATION -- random instances, integer engine vs rational engine")
141 POOLS = [[(1, 0), (0, 1), (1, 1)],
142           [(1, 0), (0, 1), (1, 1), (1, 2)],
143           [(1, 0), (0, 1), (1, 1), (1, 2), (2, 1), (1, 3)]]
144 DIMS = [[3], [4], [5], [6], [2, 2], [2, 3], [3, 2], [2, 4], [2, 2, 2]]
145 random.seed(20260831)
146 N = 6000
147 dis = 0; falseneg = 0; forced = 0; t0 = time.time()
148 for trial in range(N):
149     dims = random.choice(DIMS); POOL = random.choice(POOLS)
150     V, idx, slots = tower(dims); nv = len(V)
151     slotlist = [(i, p) for (i, pref, tail) in slots for p in pref]
152     lab = {s: random.choice([(0, 0)] + POOL) for s in slotlist}
153     edges, m = build_edges(dims, V, idx, slots, lab)
154     r = random.randint(0, min(6, 2*nv))
155     gens = [(random.randrange(nv), random.choice(POOL)) for _ in range(r)]
156     t = random.randint(0, max(0, nv-2))
157     T0 = set(random.sample(range(nv), t))
158     a = forces(nv, edges, gens, T0)
159     b = forces_Q(nv, edges, gens, T0)
160     if a: forced += 1
161     if a != b:
162         dis += 1
163         if (not a) and b:
164             falseneg += 1
165             print(" FALSE NEGATIVE dims=%s labels=%s gens=%s T0=%s"
166                   % (dims, lab, gens, sorted(T0)))
167         else:
168             print(" FALSE POSITIVE dims=%s labels=%s gens=%s T0=%s"
169                   % (dims, lab, gens, sorted(T0)))
170     if (trial+1) % 1500 == 0:
171         print(" %d/%d disagreements=%d (completing so far: %d) %.0fs"
172               % (trial+1, N, dis, forced, time.time()-t0))
173 print(" RESULT: %d instances, %d completed under the integer engine,"
174       % (N, forced))
175 print(" %d disagreements, %d of them false negatives." % (dis, falseneg))
176
177 print("\nTHIRD ENGINE -- naive integer brute force on tiny instances")
178 random.seed(7)
179 checked = 0; bad = 0
180 while checked < 400:
181     dims = random.choice([[2], [3], [2, 2]])
182     POOL = random.choice(POOLS)
183     V, idx, slots = tower(dims); nv = len(V)
184     slotlist = [(i, p) for (i, pref, tail) in slots for p in pref]
185     lab = {s: random.choice([(0, 0)] + POOL) for s in slotlist}
186     edges, m = build_edges(dims, V, idx, slots, lab)
187     gens = [(random.randrange(nv), random.choice(POOL))
188             for _ in range(random.randint(0, 4))]
189     T0 = set()
190     c = forces_brute(nv, edges, gens, T0, box=2)
191     if c is None: continue
192     checked += 1
193     a = forces(nv, edges, gens, T0)
194     if a != c:
195         bad += 1
196         print(" MISMATCH vs brute force: dims=%s labels=%s gens=%s"
197               % (dims, lab, gens))
198 print(" RESULT: %d tiny instances, %d mismatches against brute force."
199       % (checked, bad))

```

## A.5 AK-LOCALRANK-CONTROL-01 — the per-vertex control

```

1 # SCRIPT: AK-LOCALRANK-CONTROL-01
2 # The control behind the sentence "no completing object violates the per-vertex
3 # condition". Random five-vertex towers, three-slope labels, random generator
4 # sets; every completing object is checked against the corollary
5 #     every free vertex has rank 2 from {its generators} u {its incident labels}.
6 # A single violation would falsify Theorem 1. Prints the counts it quotes.
7 import random, itertools
8 exec(open('AK-COVER-SEARCH-01.py').read().split('# ----- 4. controls')[0])
9 X3 = [(1,0),(0,1),(1,1)]
10 def violations(nv, edges, gens):
11     inc = [[] for _ in range(nv)]
12     for (i,j,x) in edges: inc[i].append(x); inc[j].append(x)
13     own = [[] for _ in range(nv)]
14     for (v,x) in gens: own[v].append(x)
15     return [v for v in range(nv) if rank_of(inc[v]+own[v]) < 2]
16 random.seed(7)
17 tested = completed = bad = 0
18 V, idx, slots = tower([5]); slotlist = [(i,p) for (i,pref,tail) in slots for p in pref]
19 for _ in range(4000):
20     lab = {s: random.choice([(0,0)]+X3) for s in slotlist}
21     edges, m = build_edges([5], V, idx, slots, lab)
22     r = random.randint(2,6)
23     gens = random.sample([(v,x) for v in range(5) for x in X3], r)
24     tested += 1
25     if forces(5, edges, gens, set()):
26         completed += 1
27     if violations(5, edges, gens): bad += 1
28 print("random 5-path objects tested: %d    completed: %d    completing objects violating the per-vertex condition: %d"
29       % (tested, completed, bad))

```

## A.6 AK-RECORD-HUNT-01 and its driver

```

1 # SCRIPT: AK-RECORD-HUNT-01
2 # -----
3 # THE CELL. n=6 (both ladder orientations), all seven edges labelled (m=7),
4 # T empty, |R|=3. Score = (7+3)/6 = 5/3 = 1.6667 <= 1.675. A single completing
5 # object here IS a new arithmetic-Kakeya record. Katz-Tao's figure 2 sits in this
6 # cell with |R|=4; the question is whether a richer dilate set lets one generator
7 # be removed.
8 #
9 # WHY THIS CELL AND NOT A BIGGER GRAPH. Under tower products edge density m/n
10 # ADDS, so iterating never helps; the only amortisation is |R| bounded while n
11 # grows, i.e. more dilates. (Proved 2026-08-31.) Small n, richer X, fewer
12 # generators: this is the axis that can pay.
13 #
14 # THE PRUNE THAT MAKES IT FINITE. SEEDING LEMMA: from T empty, the first vertex
15 # to fall needs rank 2 from its own generators alone -- every incident
16 # edge has an unknown far endpoint and is annihilated by the restriction. So
17 # with |R|=3, two generators sit at one vertex with independent labels, and the
18 # third is anywhere. R = {(e,x),(e,y)} u {(f,z)}, x,y independent.
19 #
20 # SYMMETRY. M in GL2(Z) fixing the forbidden line carries any level-1 dilate
21 # to (1,0), so a rung at level 1 is WLOG (1,0). Rungs at other levels are a
22 # separate run (stated scope).
23 #
24 # KILL CONDITIONS, fixed before the run:
25 # * the engine must still verify Katz-Tao figure 2 at 11/6 (control);
26 # * with |R|=4 the search over the SAME space must recover an 11/6 object
27 #   (positive control -- an empty |R|=3 result is meaningless otherwise);
28 # * any hit is re-verified by the independent rational engine before it is
29 #   believed, and printed in Epoch's six-line submission format.
30 # -----
31 import itertools, time, sys
32 from fractions import Fraction

```

```

33 exec(open('AK-COVER-SEARCH-01.py').read().split('# ----- 4. controls')[0])
34
35 def indep(x, y): return x[0]*y[1] - x[1]*y[0] != 0
36
37 def dilates(h):
38     """primitive (p,q), |p|,|q|<=h, p+q>=1, up to sign of the pair"""
39     from math import gcd
40     out = []
41     for p in range(-h, h+1):
42         for q in range(-h, h+1):
43             if p + q >= 1 and gcd(abs(p), abs(q)) == 1:
44                 out.append((p, q))
45     return out
46
47 def submission(X, dims, labels, T, gens, V):
48     """Epoch six-line format"""
49     m = sum(1 for _ in range(0)) # placeholder, recomputed below
50     lines = []
51     lines.append("score=%s m=%d |R|=%d n=%d |T|=%d")
52     return lines
53
54 def hunt(dims, D, rung, rmax, t1, chunk=None, positive_control=False, only_rails=None):
55     V, idx, slots = tower(dims); nv = len(V)
56     slotlist = [(i, p) for (i, pref, tail) in slots for p in pref]
57     # level-0 slab = the rung (one prefix), the rest are rails
58     rung_slot = slotlist[0]; rails = slotlist[1:]
59     t0 = time.time(); calls = 0; done = True; hits = []
60     rail_choices = list(itertools.product(D, repeat=len(rails)))
61     if chunk is not None:
62         rail_choices = [c for c in rail_choices if c[0] == D[chunk]]
63     if only_rails is not None:
64         rail_choices = [only_rails]
65     allgens = [(v, x) for v in range(nv) for x in D]
66     for rc in rail_choices:
67         if time.time() - t0 > t1: done = False; break
68         labels = {rung_slot: rung}
69         labels.update(dict(zip(rails, rc)))
70         edges, m = build_edges(dims, V, idx, slots, labels)
71         # seeding lemma: some vertex carries two independent generators
72         for e in range(nv):
73             for x, y in itertools.combinations(D, 2):
74                 if not indep(x, y): continue
75                 base = [(e, x), (e, y)]
76                 extra = rmax - 2
77                 for add in itertools.combinations([g for g in allgens if g not in base], extra):
78                     gens = base + list(add)
79                     calls += 1
80                     if forces(nv, edges, gens, set()):
81                         hits.append((dims, labels, gens, m))
82                         print(" *** HIT *** dims=%s labels=%s gens=%s score=%s"
83                               % (dims, {k: v for k, v in labels.items()}, [(V[g], x) for g, x in gens],
84                                  Fraction(m + len(gens), nv)))
85                         if not positive_control: return hits, calls, done
86     return hits, calls, done
87
88 if __name__ == "__main__":
89     D = dilates(2)
90     print("dilate set (height <= 2, primitive, p+q >= 1):", D, " size", len(D))
91
92     print("\nCONTROL 1 -- engine verifies Katz-Tao figure 2 at 11/6")
93     V, idx, slots = tower([2, 3])
94     lab = {(0, (1,)): (1, 0), (1, (1, 1)): (0, 1), (1, (1, 2)): (1, 1),
95           (1, (2, 1)): (0, 1), (1, (2, 2)): (0, 1)}
96     e, m = build_edges([2, 3], V, idx, slots, lab)
97     g = [(idx[(1, 2)], (1, 0)), (idx[(2, 1)], (1, 1)), (idx[(1, 1)], (1, 1)), (idx[(1, 3)], (0, 1))]
98     print(" complete=%s score=%s" % (forces(6, e, g, set()), Fraction(m + 4, 6)))
99
100    print("\nCONTROL 2 -- with |R|=4 the SAME search must find an 11/6 object")

```



```

101 t0 = time.time()
102 h, c, dn = hunt([2, 3], D, (1, 0), 4, 600, chunk=None, only_rails=((0,1),(1,1),(0,1),(0,1)))
103 print(" found %d object(s) in %d calls, %.0fs -> %s" % (len(h), c, time.time()-t0,
104 "PASS" if h else "FAIL -- search cannot see a known object"))
105
106 mode = sys.argv[1] if len(sys.argv) > 1 else "23"
107 ch = int(sys.argv[2]) if len(sys.argv) > 2 else None
108 t1 = float(sys.argv[3]) if len(sys.argv) > 3 else 600
109 dims = [2, 3] if mode == "23" else [3, 2]
110 print("\nTHE HUNT -- d=%s, rung (1,0), rails from D, |R|=3, chunk=%s" % (dims, ch))
111 t0 = time.time()
112 h, c, dn = hunt(dims, D, (1, 0), 3, t1, chunk=ch)
113 print(" result: %s calls=%d %s %.0fs"
114 % ("RECORD FOUND" if h else "nothing completes", c,
115 "COMPLETE" if dn else "TIMEOUT", time.time()-t0))

```

```

1 import sys,time,itertools,subprocess
2 exec(open('AK-RECORD-HUNT-01.py').read().split('if __name__')[0])
3 D=dilates(2); dims=[2,3] if sys.argv[1]=="23" else [3,2]
4 start=int(sys.argv[2]); budget=float(sys.argv[3])
5 V,idx,slots=tower(dims); nv=len(V)
6 slotlist=[(i,p) for (i,pref,tail) in slots for p in pref]
7 rung_slot=slotlist[0]; rails=slotlist[1:]
8 allgens=[(v,x) for v in range(nv) for x in D]
9 pairs=[(x,y) for x,y in itertools.combinations(D,2) if indep(x,y)]
10 T0=time.time(); k=start; total=0; hits=[]
11 while k<49 and time.time()-T0<budget:
12     c1,c2=divmod(k,7); t0=time.time(); calls=0
13     for rc in itertools.product(D,repeat=len(rails)):
14         if rc[0]!=D[c1] or rc[1]!=D[c2]: continue
15         labels={rung_slot:(1,0)}; labels.update(dict(zip(rails,rc)))
16         edges,m=build_edges(dims,V,idx,slots,labels)
17         for e in range(nv):
18             for x,y in pairs:
19                 base=[(e,x),(e,y)]
20                 for add in allgens:
21                     if add in base: continue
22                     gens=base+[add]; calls+=1
23                     if forces(nv,edges,gens,set()):
24                         hits.append((labels,gens))
25                         print(" *** HIT *** labels=%s gens=%s"%(labels,[V[g],x) for g,x in gens]),flush=True)
26     total+=calls
27     print(" sub-chunk %2d (%s,%s): %s %d calls %.0fs"%(k,D[c1],D[c2],"RECORD" if hits else "nothing",calls,time.time()-t0),f
28     k+=1
29 print("d=%s: sub-chunks %d..%d done, next=%d, total calls %d, %s"%(dims,start,k-1,k,total,"RECORD FOUND" if hits else "nothing

```

## A.7 SD-ENTROPY-LP-01 — the Shannon LP

```

1 # SCRIPT: SD-ENTROPY-LP-01
2 # -----
3 # THE SWING. Ruzsa's equivalence says the sum-difference problem IS an entropy
4 # inequality:  $SD(\{0,1,\text{inf}\};-1)$  is the least  $C$  with
5 #  $H(X-Y) \leq C * \max(H(X), H(Y), H(X+Y))$ 
6 # for all discrete  $(X,Y)$ . The record  $C = 11/6$  is from 1999 and Tao states
7 # plainly that AlphaEvolve could not be made to produce UPPER bounds at all.
8 #
9 # Upper bounds of this shape are exactly Shannon-type inequalities, and whether
10 # one is Shannon-type is decidable by a linear program over the elemental
11 # inequalities (Yeung's ITIP method). Nobody appears to have pointed that
12 # machinery at this constant. So: point it.
13 #
14 # WHAT THE LP IS. Unknowns  $h(S)$  for every nonempty subset  $S$  of a ground set of
15 #  $k$  random variables. Constraints:
16 # (i) elemental Shannon inequalities -- monotonicity  $h([k]) \geq h([k] \setminus i)$ ,
17 # and submodularity  $h(S+i)+h(S+j) \geq h(S+i+j)+h(S)$ ;
18 # (ii) structure: functional dependence  $h(S \cup T)=h(S)$  when  $T$  is a function of
19 #  $S$ , independence  $h(S \cup T)=h(S)+h(T)$ , identical distribution.
20 # Maximising  $h(\text{target})$  subject to the three control entropies  $\leq 1$  gives the
21 # best constant provable from Shannon inequalities alone with these variables.
22 #
23 # KILL CONDITIONS, FIXED BEFORE RUNNING:
24 # CONTROL 1 must return exactly 2 (the trivial bound) on the bare four
25 # projections -- if it returns less, the LP is unsound; more, and the
26 # structure constraints are wrong.
27 # CONTROL 2 must certify Ruzsa's entropy triangle inequality
28 #  $H(X-Z) \leq H(X-Y) + H(Y-Z) - H(Y)$  for independent  $X,Y,Z$ , which IS
29 # Shannon-type. If the engine cannot reproduce a known theorem it cannot
30 # be trusted on an unknown one.
31 # Only then does the two-copy run mean anything.
32 #
33 # HONEST PRIOR, WRITTEN BEFORE THE RUN: I expect 2 from the one-copy setup, and
34 # I expect the two-copy setup to fall short of 11/6, because Katz-Tao's proof
35 # uses Balog-Szemerédi-Gowers, which is NOT a Shannon-type inequality. A result
36 # strictly below 2 would already be new information about how far pure Shannon
37 # reasoning reaches on this constant.
38 # -----
39
40 import itertools, numpy as np
41 from scipy.optimize import linprog
42 from fractions import Fraction
43
44 def solve(k, names, equalities, caps, target, extra_ub=None, verbose=True):
45     """Maximise  $h(\text{target})$  s.t. elemental Shannon inequalities, the supplied
46     equalities, and  $h(c) \leq 1$  for each  $c$  in caps. Subsets are bitmasks."""
47     N = 1 << k
48     idx = {m: m-1 for m in range(1, N)} # unknown vector position
49     nvar = N - 1
50     A_ub, b_ub, A_eq, b_eq = [], [], [], []
51
52     def row():: return np.zeros(nvar)
53     def add(r, m, c):
54         if m: r[idx[m]] += c
55
56     full = N - 1
57     # monotonicity:  $h(\text{full}) - h(\text{full} \setminus i) \geq 0 \rightarrow -h(\text{full}) + h(\text{full} \setminus i) \leq 0$ 
58     for i in range(k):
59         r = row(); add(r, full, -1); add(r, full & ~(1 << i), 1)
60         A_ub.append(r); b_ub.append(0.0)
61     # submodularity:  $h(S+i) + h(S+j) - h(S+i+j) - h(S) \geq 0$ 
62     for i in range(k):
63         for j in range(i+1, k):
64             rest = [t for t in range(k) if t != i and t != j]
65             for bits in range(1 << len(rest)):
66                 S = 0

```

```

67         for p, t in enumerate(rest):
68             if bits >> p & 1: S |= (1 << t)
69         r = row()
70         add(r, S | (1 << i), -1); add(r, S | (1 << j), -1)
71         add(r, S | (1 << i) | (1 << j), 1); add(r, S, 1)
72         A_ub.append(r); b_ub.append(0.0)
73     # supplied equalities: list of (list_of_(mask,coef), rhs)
74     for terms, rhs in equalities:
75         r = row()
76         for m, c in terms: add(r, m, c)
77         A_eq.append(r); b_eq.append(float(rhs))
78     # normalisation caps
79     for m in caps:
80         r = row(); add(r, m, 1)
81         A_ub.append(r); b_ub.append(1.0)
82     if extra_ub:
83         for terms, rhs in extra_ub:
84             r = row()
85             for m, c in terms: add(r, m, c)
86             A_ub.append(r); b_ub.append(float(rhs))
87
88     c = row(); add(c, target, -1) # maximise -> minimise -obj
89     res = linprog(c, A_ub=np.array(A_ub), b_ub=np.array(b_ub),
90                 A_eq=np.array(A_eq) if A_eq else None,
91                 b_eq=np.array(b_eq) if b_eq else None,
92                 bounds=[(0, None)]*nvar, method="highs")
93     if verbose:
94         print("    LP: %d unknowns, %d inequalities, %d equalities -> %s"
95               % (nvar, len(A_ub), len(A_eq), res.message.strip()))
96     return (-res.fun if res.success else None)
97
98 def bit(i): return 1 << i
99
100 # -----
101 print("CONTROL 1 | the four projections of one pair. Must return exactly 2.")
102 # ground: 0=X, 1=Y, 2=X+Y, 3=X-Y. Any TWO of them determine the pair, so every
103 # subset of size >= 2 has the same entropy as the full set.
104 k = 4
105 eqs = []
106 full = (1 << k) - 1
107 for m in range(1, 1 << k):
108     if bin(m).count("1") >= 2 and m != full:
109         eqs.append([(m, 1), (full, -1)], 0))
110 v = solve(k, "XY+-", eqs, [bit(0), bit(1), bit(2)], bit(3))
111 print("    best constant provable from Shannon alone = %.6f (expected 2)" % v)
112 print("    -> %s\n" % ("PASS" if abs(v-2) < 1e-7 else "FAIL | engine unsound"))
113
114 # -----
115 print("CONTROL 2 | Ruzsa's entropy triangle inequality, a KNOWN Shannon-type result.")
116 # ground: 0=X 1=Y 2=Z 3=P=X-Y 4=Q=Y-Z 5=R=X-Z ; X,Y,Z mutually independent.
117 k = 6
118 X, Y, Z, P, Q, R = [bit(i) for i in range(6)]
119 eqs = []
120 eqs.append([(X | Y | Z, 1), (X, -1), (Y, -1), (Z, -1)], 0)) # independence
121 for base, fn in [(X | Y, P), (Y | Z, Q), (X | Z, R), (P | Q, R),
122                 (X | P, Y), (Y | Q, Z), (X | R, Z)]:
123     eqs.append([(base, 1), (base, -1)], 0)) # functional dep
124 # everything is a function of (X,Y,Z)
125 allv = (1 << k) - 1
126 eqs.append([(allv, 1), (X | Y | Z, -1)], 0))
127 # Ruzsa says h(R) - h(P) - h(Q) + h(Y) <= 0. Maximise it under a scale cap.
128 res = solve(k, "", eqs, [], R, extra_ub=[[(allv, 1)], 1])
129 # maximise the combination directly:
130 def solve_comb(k, eqs, obj_terms, cap_terms):
131     N = 1 << k; nvar = N-1
132     A_ub, b_ub, A_eq, b_eq = [], [], [], []
133     def row(): return np.zeros(nvar)
134     def add(r, m, c):

```

```

135         if m: r[m-1] += c
136     full = N-1
137     for i in range(k):
138         r = row(); add(r, full, -1); add(r, full & ~(1 << i), 1)
139         A_ub.append(r); b_ub.append(0.0)
140     for i in range(k):
141         for j in range(i+1, k):
142             rest = [t for t in range(k) if t != i and t != j]
143             for bits in range(1 << len(rest)):
144                 S = 0
145                 for p, t in enumerate(rest):
146                     if bits >> p & 1: S |= (1 << t)
147                 r = row()
148                 add(r, S | (1<<i), -1); add(r, S | (1<<j), -1)
149                 add(r, S | (1<<i) | (1<<j), 1); add(r, S, 1)
150                 A_ub.append(r); b_ub.append(0.0)
151     for terms, rhs in eqs:
152         r = row()
153         for m, c in terms: add(r, m, c)
154         A_eq.append(r); b_eq.append(float(rhs))
155     r = row()
156     for m, c in cap_terms: add(r, m, c)
157     A_ub.append(r); b_ub.append(1.0)
158     c = row()
159     for m, co in obj_terms: add(c, m, -co)
160     out = linprog(c, A_ub=np.array(A_ub), b_ub=np.array(b_ub),
161                 A_eq=np.array(A_eq), b_eq=np.array(b_eq),
162                 bounds=[(0, None)]*nvar, method="highs")
163     return -out.fun if out.success else None
164 val = solve_comb(6, eqs, [(R, 1), (P, -1), (Q, -1), (Y, 1)], [(allv, 1)])
165 print("    max of  H(X-Z) - H(X-Y) - H(Y-Z) + H(Y) = %.9f    (Ruzsa: must be <= 0)" % val)
166 print("    -> %s\n" % ("PASS" if val < 1e-7 else "FAIL | engine cannot see a known theorem"))
167
168 # -----
169 print("THE RUN | two independent copies, with cross differences.")
170 # ground: 0=X1 1=Y1 2=S1=X1+Y1 3=D1=X1-Y1 4=X2 5=Y2 6=E=X1-Y2 7=F=X2-Y1
171 k = 8
172 X1, Y1, S1, D1, X2, Y2, E, Fv = [bit(i) for i in range(8)]
173 eqs = []
174 P1 = X1 | Y1; P2 = X2 | Y2
175 # within copy 1: any two of {X1,Y1,S1,D1} determine the pair
176 for m in range(1, 1 << 4):
177     if bin(m).count("1") >= 2:
178         eqs.append([(m, 1), (P1, -1)], 0))
179 # copies independent and identically distributed
180 eqs.append([(P1 | P2, 1), (P1, -1), (P2, -1)], 0))
181 eqs.append([(X1, 1), (X2, -1)], 0))
182 eqs.append([(Y1, 1), (Y2, -1)], 0))
183 eqs.append([(P1, 1), (P2, -1)], 0))
184 # cross variables are functions of their pairs
185 eqs.append([(X1 | Y2 | E, 1), (X1 | Y2, -1)], 0))
186 eqs.append([(X2 | Y1 | Fv, 1), (X2 | Y1, -1)], 0))
187 eqs.append([(X1 | Y2, 1), (X1, -1), (Y2, -1)], 0)) # X1 _||_ Y2
188 eqs.append([(X2 | Y1, 1), (X2, -1), (Y1, -1)], 0))
189 eqs.append([(E, 1), (Fv, -1)], 0)) # same distribution
190 # everything is a function of the two pairs
191 allv = (1 << k) - 1
192 eqs.append([(allv, 1), (P1 | P2, -1)], 0))
193 v2 = solve(k, "", eqs, [X1, Y1, S1], D1)
194 print("    best constant from Shannon + two copies = %.6f" % v2)
195 print("    record to beat: 11/6 = %.6f ; trivial bound: 2" % (11/6))

```

## References

- [1] Epoch AI. *The Arithmetic Kakeya Conjecture*. FrontierMath: Open Problems, 2026. <https://epoch.ai/frontiermath/open-problems/arithmetic-kakeya>
- [2] A. S. Besicovitch. On Kakeya’s problem and a similar one. *Math. Z.* 27 (1928), 312–320.
- [3] J. Bourgain. On the dimension of Kakeya sets and related maximal inequalities. *Geom. Funct. Anal.* 9 (1999), 256–282.
- [4] B. Green, I. Z. Ruzsa. On the arithmetic Kakeya conjecture of Katz and Tao. *Period. Math. Hungar.* 78 (2019), 135–151.
- [5] N. H. Katz. Elementary proofs and the sums differences problem. *Collect. Math.* 57 Extra (2006), 275–280.
- [6] N. H. Katz, T. Tao. Bounds on arithmetic projections, and applications to the Kakeya conjecture. *Math. Res. Lett.* 6 (1999), 625–630.
- [7] N. H. Katz, T. Tao. New bounds for Kakeya problems. *J. Anal. Math.* 87 (2002), 231–263.
- [8] M. Lemm. New counterexamples for sums-differences. *Proc. Amer. Math. Soc.* 143 (2015), 3863–3868.
- [9] C. Pohoata, D. Zakharov. *Generalized Arithmetic Kakeya*. arXiv:2411.13395, 2024.
- [10] T. Tao. *Sum-difference exponents for boundedly many slopes, and rational complexity*. arXiv:2511.15135, 2025.